



UNIVERSITÀ DEGLI STUDI DI GENOVA
DIBRIS – DEPARTMENT OF INFORMATICS, BIOENGINEERING, ROBOTICS AND
SYSTEMS ENGINEERING

Assignment D3-V1

Warehouse Robotics

Single Robot Pick-and-Deliver

Artificial Intelligence for Robotics II – 104731

Author:
Paolo NICOLINI
s5698969

Professors:
Prof. Fulvio MASTROGIOVANNI
Prof. Omar KASHMAR

19 May 2026

Contents

1	Introduction and Scenario	2
1.1	Domain Characteristics	2
1.2	Modelling Guidelines	2
2	The Warehouse Environment	3
3	Q1 – Classical PDDL Model	3
3.1	Domain Specification	3
3.2	Actions	3
3.3	Classical Problem 1 – Single Package	4
3.4	Classical Problem 2 – Sequential Delivery	4
4	Q2 – PDDL+ Extension	4
4.1	Continuous Transportation	4
4.2	Continuous Package Aging	5
4.3	Deadline Breach Event	5
4.4	State Transitions	5
4.5	PDDL+ Problem Instances	6
5	Experimental Results	6
5.1	Summary	6
5.2	Classical Plans	6
5.3	PDDL+ Plans	7
6	Discussion	7
6.1	Scalability from Single to Multiple Packages	8
6.2	Limitations of Purely Sequential Planning	8
6.3	Heuristic Blind Spots and Delete Relaxation	8
7	Conclusion	8
7.1	Limitations of the Current Model	9
7.2	Future Work	9

1. Introduction and Scenario

This project addresses Assignment **D3-V1: Warehouse Robotics – Single Robot Pick-and-Deliver**, in which a mobile robot operates inside a warehouse composed of storage aisles and shipping zones. The robot is required to pick up packages from storage areas and transport them to specified delivery destinations.

The system models a strongly constrained agent: **the robot can carry only one package at a time** and must navigate between locations exclusively through predefined connections of the warehouse graph. These constraints reflect realistic single-arm pick-and-place manipulators commonly deployed in modern logistics environments.

The work is divided into two parts:

- A **Classical PDDL model** that captures the discrete pick-move-deliver logic and validates the influence of the single-package capacity constraint on the resulting plans.
- A **PDDL+ extension** that introduces *continuous numeric fluents, processes and events* to model travel time and strict delivery deadlines, transforming the planning problem into a hybrid discrete-continuous one.

The main objective is to demonstrate how limited carrying capacity and time-dependent delivery constraints jointly influence the feasibility and structure of the generated plans.

1.1 Domain Characteristics

The domain is characterized by the following elements:

- **Robot:** a single mobile manipulator with position and hand state.
- **Environment:** a graph of warehouse locations (dock, storage aisles, shipping bay).
- **Tasks:** explicit *pick*, *transport*, and *deliver* operations.
- **Constraints:** strict single-package capacity; in the PDDL+ extension, also strict delivery deadlines.
- **Planning Tool:** ENHSP-20 planner with $h^{\max}$ heuristic for PDDL+ problems.

1.2 Modelling Guidelines

The model adheres to the guidelines specified in the assignment:

- Robot position and package location are modelled explicitly via dedicated predicates.
- Pick and drop actions are represented separately rather than being fused.
- Movement is never implicit: each displacement requires an explicit **move** (or **start-move/end-move**) action.
- Carrying constraints (**hand-empty**, **holding**) actively shape the planner's decisions and force sequential delivery.
- All continuous behaviour in PDDL+ is declarative: processes and events are defined in the domain, not hardcoded in the planner.

2. The Warehouse Environment

The warehouse is modelled as an undirected graph $G = (V, E)$, where V is the set of locations and E the set of bidirectional connections traversable by the robot. Figure 1 shows the topology used in the most complex scenarios (Classical Problem 2 and PDDL+ Problem 2).

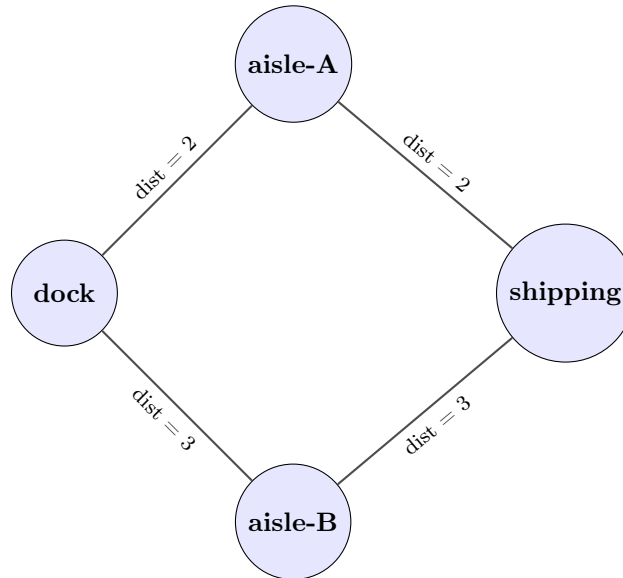


Figure 1: Warehouse topology for the complex scenarios. Distances refer to the PDDL+ formulation and represent time units for travel.

The graph features two parallel routes from **dock** to **shipping**, each passing through one storage aisle. This topology forces the planner to explicitly choose visitation orders, exposing the impact of the single-capacity constraint.

3. Q1 – Classical PDDL Model

3.1 Domain Specification

The classical domain **warehouse-single-robot** defines three primary types — **robot**, **package**, **location** — and five core predicates:

```

1 (at-robot ?r - robot ?l - location)
2 (at-package ?p - package ?l - location)
3 (connected ?l1 - location ?l2 - location)
4 (holding ?r - robot ?p - package)
5 (hand-empty ?r - robot)

```

Listing 1: Predicates of the classical domain.

The robot’s physical constraint is enforced by the pair **hand-empty/holding**. Since a successful **pick** requires **hand-empty** and produces **holding**, no second package can ever be lifted before the first one is dropped.

3.2 Actions

Three actions describe the discrete behaviour of the robot:

- **move** – transports the robot between two connected locations in a single step.

- **pick** – picks up a package, provided the hand is empty and both robot and package share the same location.
- **drop** – releases the package at the current location, enabling subsequent deliveries or intermediate storage.

```

1 (:action pick
2   :parameters (?r - robot ?p - package ?l - location)
3   :precondition (and
4     (at-robot ?r ?l)
5     (at-package ?p ?l)
6     (hand-empty ?r)
7   )
8   :effect (and
9     (not (at-package ?p ?l))
10    (not (hand-empty ?r))
11    (holding ?r ?p)
12  )
13 )

```

Listing 2: The pick action enforcing the single-capacity constraint.

3.3 Classical Problem 1 – Single Package

problem-1-single-package.pddl contains:

- one robot (robby) starting at the dock;
- one package pkg1 stored in aisle-A;
- three locations: dock, aisle-A, shipping;
- the goal (at-package pkg1 shipping).

This minimal instance validates the elementary sequence *move* → *pick* → *move* → *drop*.

3.4 Classical Problem 2 – Sequential Delivery

problem-2-sequential-delivery.pddl models the topology of Figure 1 with three packages: pkg1 and pkg2 stored in aisle-A, and pkg3 stored in aisle-B. All three must be delivered to shipping.

Because the robot can carry only one package at a time, the planner is forced to alternate between storage aisles and the shipping bay, producing a *back-and-forth* plan that explicitly demonstrates the influence of the single-capacity constraint.

4. Q2 – PDDL+ Extension

The PDDL+ domain `warehouse-continuous-robot` extends the classical model with two continuous processes and one automatic event, transforming the planning problem from purely symbolic to *hybrid discrete-continuous*.

4.1 Continuous Transportation

Movement is no longer instantaneous: it is split into **start-move** and **end-move**. While the robot is in transit, a continuous process `robot-transit` decreases the residual distance to the target proportionally to the elapsed time.

```

1 (:process robot-transit
2   :parameters (?r - robot)
3   :precondition (and (is-moving ?r) (> (distance-left ?r) 0))
4   :effect (decrease (distance-left ?r) (* #t 1))
5 )

```

Listing 3: The continuous transportation process.

The **end-move** action becomes applicable only when $(\text{distance-left } ?r) \leq 0$, faithfully representing physical travel time. This design decouples movement into two phases: a discrete commitment phase (**start-move**) and a completion phase (**end-move**) that depends on continuous progress.

4.2 Continuous Package Aging

Every undelivered package carries a strict deadline encoded in the numeric fluent (**time-remaining ?p**). A second process, **package-aging**, continuously decreases this value:

```

1 (:process package-aging
2   :parameters (?p - package)
3   :precondition (and (not (delivered ?p)) (> (time-remaining ?p) 0))
4   :effect (decrease (time-remaining ?p) (* #t 1))
5 )

```

Listing 4: The package aging process.

4.3 Deadline Breach Event

When a package’s residual time reaches zero before delivery, an automatic event fires and irreversibly marks the package as missed:

```

1 (:event deadline-breach
2   :parameters (?p - package)
3   :precondition (and
4     (not (delivered ?p))
5     (<= (time-remaining ?p) 0)
6     (not (missed-deadline ?p))
7   )
8   :effect (missed-deadline ?p)
9 )

```

Listing 5: The deadline-breach event.

The third precondition (**not (missed-deadline ?p)**) is crucial: without it, the event would fire infinitely many times whenever **time-remaining** stays at zero, sending ENHSP into an infinite loop. Including it guarantees the event fires *exactly once* per package.

Once (**missed-deadline ?p**) becomes true, the **deliver-package** action becomes permanently inapplicable, since its precondition requires the negation of that very predicate.

4.4 State Transitions

Figure 2 summarises the lifecycle of a package under the PDDL+ formulation.

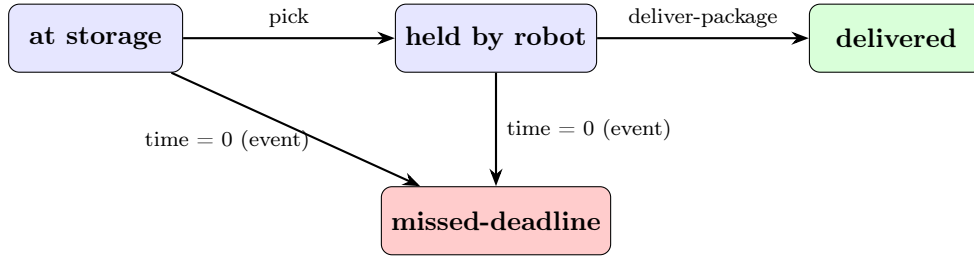


Figure 2: Lifecycle of a package in the PDDL+ model. The continuous processes (robot-transit and package-aging) evolve independently, making the deadline a hard constraint.

4.5 PDDL+ Problem Instances

Problem Plus 1 – Simple Delivery. A minimal scenario with one package and unit distances, intended to verify the correctness of the continuous time mechanics on a trivial topology.

Expected result: Problem Solved.

Problem Plus 2 – Complex Balanced. Three packages with independent aging clocks. Distances and deadlines were carefully tuned ($d \in \{2, 3\}$, deadlines $\in \{30, 45, 60\}$) to keep the branching factor mathematically tractable for the heuristic while still requiring an optimised route.

Expected result: Problem Solved.

Problem Plus 3 – Missed Deadline. A scenario explicitly designed to be infeasible. The travel time required to reach the package and deliver it (at least 4 time units) strictly exceeds the package’s initial lifespan (`time-remaining = 2`). The `deadline-breach` event always fires before delivery becomes possible.

Expected result: Problem unsolvable.

5. Experimental Results

All experiments were carried out on Ubuntu (WSL) with Java 21 and the **ENHSP-20** planner. The classical problems were solved using ENHSP’s default search strategy, while the PDDL+ problems used the `opt-hrmax` heuristic to handle numeric fluents effectively.

5.1 Summary

Table 1: Summary of experimental results. All experiments completed successfully without errors or infinite loops.

Problem	Model	Result	Plan Length	Search Time
Problem 1	Classical	Solved	4	~ 0.01 s
Problem 2	Classical	Solved	12	~ 0.05 s
Problem Plus 1	PDDL+	Solved	8	~ 0.06 s
Problem Plus 2	PDDL+	Solved	32	~ 1.00 s
Problem Plus 3	PDDL+	Unsolvable	–	~ 0.00 s

5.2 Classical Plans

For Problem 1, ENHSP returned a 4-step plan validating the basic operational sequence:

```

1 0.0: (move  robby dock      aisle-A)
2 1.0: (pick  robby pkg1     aisle-A)
3 2.0: (move  robby aisle-A shipping)
4 3.0: (drop  robby pkg1     shipping)

```

Listing 6: Plan for Classical Problem 1. The robot simply moves, picks, moves again, and drops.

For Problem 2, the planner produced a 12-step back-and-forth plan delivering all three packages. The structure clearly reflects the single-capacity constraint: each delivery is followed by a return to a storage aisle.

```

1 0.0: (move  robby dock      aisle-A)
2 1.0: (pick  robby pkg1     aisle-A)
3 2.0: (move  robby aisle-A shipping)
4 3.0: (drop  robby pkg1     shipping)
5 4.0: (move  robby shipping aisle-B)
6 5.0: (pick  robby pkg3     aisle-B)
7 6.0: (move  robby aisle-B shipping)
8 7.0: (drop  robby pkg3     shipping)
9 8.0: (move  robby shipping aisle-A)
10 9.0: (pick  robby pkg2     aisle-A)
11 10.0: (move robby aisle-A shipping)
12 11.0: (drop robby pkg2     shipping)

```

Listing 7: Plan for Classical Problem 2. The single-capacity constraint forces sequential delivery with alternating trips.

5.3 PDDL+ Plans

The PDDL+ plans include explicit `--waiting--` blocks corresponding to the elapsed time during which the continuous processes evolve. For Problem Plus 2, the planner expanded 23,731 nodes and evaluated 41,510 states before extracting an optimal 32-step plan:

```

1 0:      (start-move      robby dock      aisle-A)
2 0:      -----waiting---- [2.0]
3 2.0:    (end-move       robby dock      aisle-A)
4 2.0:    (pick           robby pkg2     aisle-A)
5 2.0:    (start-move     robby aisle-A shipping)
6 2.0:    -----waiting---- [4.0]
7 4.0:    (end-move       robby aisle-A shipping)
8 4.0:    (deliver-package robby pkg2     shipping)
9 ...
10 14.0:  (deliver-package robby pkg1     shipping)

```

Listing 8: Excerpt of the plan for PDDL+ Problem 2. Waiting blocks show continuous time evolution.

The total makespan was 14 time units, with all packages successfully delivered well within their respective deadlines (30, 45, 60). This demonstrates that the planner successfully balanced temporal constraints across multiple packages.

For Problem Plus 3, ENHSP correctly terminated with **Problem unsolvable** after only 5 expanded nodes and 3 detected dead-ends, confirming that the deadline constraint effectively restricts the reachable state space and that the goal constraint is correctly enforced.

Complete output files are stored in the `outputs/` folder of the project repository.

6. Discussion

6.1 Scalability from Single to Multiple Packages

During development we observed that scaling from one to multiple packages in a continuous-time environment drastically inflates the state space. Each package carries its own independent, continuously ticking deadline; therefore the number of relevant temporal interleavings grows roughly exponentially with the number of packages.

Empirically, the search effort grew from ~ 14 expanded nodes for Problem Plus 1 to $\sim 23,731$ for Problem Plus 2 (a $\sim 1700\times$ increase for only $3\times$ the packages). To keep the search tractable, the physical distances were kept small ($d \in \{2, 3\}$) and the deadlines generous (30, 45, 60). This balance was essential to allow the $h^{\max}$ heuristic to effectively prune dead-end timelines.

This exponential growth suggests that scaling to larger instances (e.g., 10+ packages) would require either much stronger heuristics, domain abstraction, or relaxation of the continuous-time model.

6.2 Limitations of Purely Sequential Planning

The single-package capacity constraint forces a strictly sequential delivery pattern. Every successful delivery is followed by an *empty-handed* return trip to one of the storage aisles. The robot cannot batch deliveries even when two packages share the same source aisle (e.g., `pkg1` and `pkg2` in `aisle-A`).

This is clearly visible in the Classical Problem 2 plan, where the 12 steps could in principle be reduced significantly with a multi-package gripper. In a real-world warehouse, this limitation would severely cap throughput, suggesting that future extensions should relax the single-capacity assumption in favour of a multi-package model or a delivery-batching mechanism.

6.3 Heuristic Blind Spots and Delete Relaxation

Problem Plus 3 was designed specifically to expose a subtle heuristic pitfall. ENHSP’s relaxed heuristic is based on *delete relaxation*: negative effects of actions and events are temporarily ignored when estimating goal distance. As a consequence, the heuristic initially considered a package with `missed-deadline = true` still deliverable.

We addressed this by explicitly embedding the failure condition into the goal:

```

1 (:goal
2   (and
3     (delivered pkg1)
4     (not (missed-deadline pkg1))
5   )
6 )

```

Listing 9: The goal explicitly forbidding deadline breaches.

With this formulation, the heuristic correctly evaluated the dead-end and ENHSP terminated immediately after only 5 expanded nodes, confirming the unsolvability of the instance. This highlights an important lesson: in PDDL+ planning, explicitly stating failure conditions in the goal can be more effective than relying on implicit unsolvability detection.

7. Conclusion

This project illustrated how symbolic and hybrid planning can represent a constrained warehouse-robotics scenario. The classical PDDL model showed how a simple capacity predicate (`hand-empty`) is sufficient to force structured sequential plans. The PDDL+ extension demonstrated how continuous processes and automatic events can model travel time and delivery deadlines in a clean and declarative way.

The experimental results confirm three key observations:

1. Single-capacity constraints inflate plan length linearly with the number of packages.
2. Continuous time and per-object deadlines inflate the state space *exponentially*, demanding careful tuning of numerical parameters.
3. Delete-relaxation heuristics may underestimate the cost of unsolvable PDDL+ instances unless failure conditions are explicitly encoded in the goal.

7.1 Limitations of the Current Model

The model does not represent:

- Multi-gripper or variable carrying capacity.
- Realistic file systems, memory fragmentation, or storage management.
- Probabilistic travel times, sensor noise, or stochastic elements.
- Multiple cooperating robots or interference between agents.
- Energy constraints, battery depletion, or recharging requirements.
- Low-level motion planning or geometric collision avoidance.

Movement is instantaneous in the classical PDDL formulation. In the PDDL+ extension, travel time is modelled as a continuous linear function proportional to distance, which abstracts away acceleration, deceleration, and terrain difficulty.

7.2 Future Work

The model provides a sound foundation that could be extended in several directions:

- **Multi-package gripping:** extend the model to allow the robot to carry $k > 1$ packages, significantly reducing makespan and exploring packing constraints.
- **Multiple cooperating robots:** introduce inter-robot scheduling and coordination to understand scalability and interference.
- **Stochastic travel times:** model uncertain transportation durations and replanning strategies for deadline maintenance.
- **Temporal flexibility:** allow soft deadlines with penalties rather than hard constraints, enabling trade-offs between cost and delay.
- **Low-level integration:** connect the abstract PDDL plans to motion primitives for geometric path planning and collision avoidance.